

使用 Cloud Build、Artifact Registry 和 GKE 安全地建構及部署

來源：<https://codelabs.developers.google.com/secure-build-deploy-cloud-build-ar-gke?hl=zh-tw>

步驟數：10

目錄

1. [簡介](#)
2. [設定和需求](#)
3. [自動掃描](#)
4. [On-Demand Scanning](#)
5. [建構管道掃描](#)
6. [簽署映像檔](#)
7. [使用 Cloud Build 簽署](#)
8. [許可控制政策](#)
9. [封鎖 GKE 中的安全漏洞](#)
10. [恭喜！](#)

1. 簡介

容器分析提供容器的安全漏洞掃描和中繼資料儲存空間。掃描服務會對 Artifact Registry 和 Container Registry 中的映像檔執行安全漏洞掃描，然後儲存產生的中繼資料，並透過 API 提供使用。中繼資料儲存空間可讓您儲存來自不同來源的資訊，包括安全漏洞掃描、Google Cloud 服務和第三方供應商。

安全漏洞掃描可以自動執行或隨選執行：

- 啟用[自動掃描](#)後，每當您將新映像檔推送到 Artifact Registry 或 Container Registry 時，系統就會自動觸發掃描作業。發現新的安全漏洞時，系統會持續更新安全漏洞資訊。
- 啟用[隨選掃描](#)後，您必須執行指令，才能掃描本機映像檔或 Artifact Registry/Container Registry 中的映像檔。隨選掃描功能可讓您彈性選擇掃描容器的時間。舉例來說，您可以掃描在本機建構的映像檔，並修復安全漏洞，然後再將映像檔儲存在登錄檔中。掃描結果最多會在掃描完成後保留 48 小時，且掃描後不會更新安全漏洞資訊。

將 Container Analysis 整合至 CI/CD pipeline 後，您就能根據中繼資料做出決策。舉例來說，您可以使用[二進位授權](#)建立部署政策，只允許從受信任的登錄檔部署符合規範的映像檔。

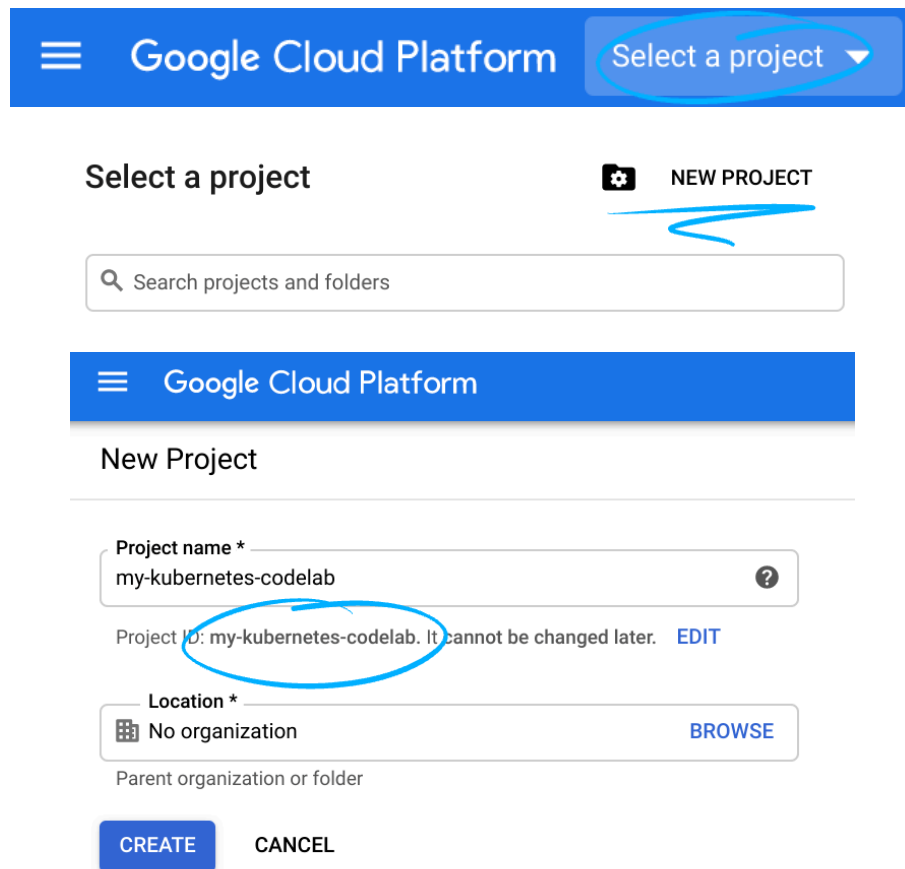
課程內容

- 如何啟用自動掃描功能
 - 如何執行 On-Demand Scanning
 - 如何在建構管道中整合掃描功能
 - 如何簽署核准的圖片
 - 如何使用 GKE 准入控制器封鎖映像檔
 - 如何設定 GKE，只允許已簽署的核准映像檔
-

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。



The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' button, which is circled in blue. Below this, the 'Select a project' section includes a search bar. The 'New Project' section follows, with a 'Project name' field containing 'my-kubernetes-codelab' (also circled in blue). Below the name field, the 'Project ID' is displayed as 'my-kubernetes-codelab' with a note that it cannot be changed later. The 'Location' is set to 'No organization'. At the bottom, there are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新該位置資訊。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生不重複的字串，通常您不需要在意這個字串。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間都會維持這個設定。
- 請注意，部分 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是專屬 ID，選定後就無法變更，且其他使用者無法使用。您是該 ID 的唯一使用者。即使刪除專案，該 ID 也無法再使用。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

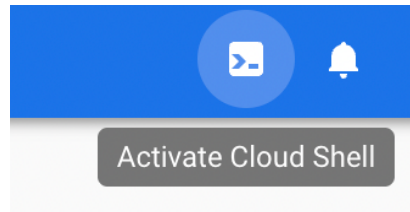
2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要關閉資源，避免產生本教學課程以外的費用，您可以刪除自己建立的資源，或刪除整個專案。Google

Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell 編輯器

本實驗室專為 Google Cloud Shell 編輯器設計及測試，如要存取編輯器，請按照下列步驟操作：

1. 前往 <https://console.cloud.google.com> 存取 Google 專案。
2. 按一下右上角的 Cloud Shell 編輯器圖示



3. 視窗底部會開啟新窗格

環境設定

在 Cloud Shell 設定專案的專案 ID 和專案編號，分別儲存為 `PROJECT_ID` 和 `PROJECT_ID` 變數。

```
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID \
  --format='value(projectNumber)')
```

啟用服務

啟用所有必要服務：

```
gcloud services enable \
  cloudkms.googleapis.com \
  cloudbuild.googleapis.com \
  container.googleapis.com \
  containerregistry.googleapis.com \
  artifactregistry.googleapis.com \
  containerscanning.googleapis.com \
  ondemandscanning.googleapis.com \
  binaryauthorization.googleapis.com
```

建立 Artifact Registry 存放區

在本實驗室中，您將使用 Artifact Registry 儲存及掃描映像檔。使用下列指令建立存放區。

```
gcloud artifacts repositories create artifact-scanning-repo \
  --repository-format=docker \
  --location=us-central1 \
  --description="Docker repository"
```

設定 Docker，在存取 Artifact Registry 時使用 gcloud 憑證。

```
gcloud auth configure-docker us-central1-docker.pkg.dev
```

步驟 3 · 約 0 分鐘

3. 自動掃描

每當您將新映像檔推送到 Artifact Registry 或 Container Registry 時，系統就會自動觸發 Artifact 掃描作業。發現新的安全漏洞時，系統會持續更新安全漏洞資訊。在本節中，您將映像檔推送至 Artifact Registry，並探索結果。

建立並變更為工作目錄

```
mkdir vuln-scan && cd vuln-scan
```

定義範例圖片

建立名為 Dockerfile 的檔案，其中含有下列內容。

```
cat > ./Dockerfile << EOF
FROM gcr.io/google-
appengine/debian9@sha256:ebffcf0df9aa33f342c4e1d4c8428b784fc571cdf6fbab0b31330347ca8af97a

# System
RUN apt update && apt install python3-pip -y

# App
WORKDIR /app
COPY . ./

RUN pip3 install Flask==1.1.4
RUN pip3 install gunicorn==20.1.0

CMD exec gunicorn --bind :$PORT --workers 1 --threads 8 --timeout 0 main:app

EOF
```

建立名為 main.py 的檔案，其中含有下列內容：

```
cat > ./main.py << EOF
import os
from flask import Flask

app = Flask(__name__)

@app.route("/")
def hello_world():
    name = os.environ.get("NAME", "Worlds")
    return "Hello {}!".format(name)

if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=int(os.environ.get("PORT", 8080)))
EOF
```

建構映像檔並推送至 AR

使用 Cloud Build 建構容器，並自動推送至 Artifact Registry。請注意圖片上的 `bad` 標記。方便您在後續步驟中識別。

```
gcloud builds submit . -t us-central1-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image:bad
```

查看映像檔詳細資料

建構程序完成後，請在 Artifact Registry 資訊主頁查看映像檔和安全漏洞結果。

- 1. 在 Cloud 控制台開啟 [Artifact Registry](#)
- 2. 點選「artifact-scanning-repo」查看內容
- 3. 點選圖片詳細資料
- 4. 點選映像檔最新的摘要
- 5. 掃描完成後，點選映像檔的安全漏洞分頁標籤

在「安全漏洞」分頁，您會看到新建映像檔的自動掃描結果。

Google Cloud

qwiklabs-gcp-04-986b59da1f76

Search (/) for resources, docs, products, and more

Search

Artifact Registry

Repositories

Settings

350d1a869d1ea

DELETE

SETUP INSTRUCTIONS

DEPLOY

REFRESH

SHOW ALL VERSIONS

us-central1-docker.pkg.dev

artifact-scanning-repo

sample-image

sha256:350d1a869d1ea96cd930417d13be6a0d9be96ab1206327a26c06034f5f192bdf

OVERVIEW

VULNERABILITIES

PULL

MANIFEST

Scan results

Based on factors such as exploitability, scope, impact, and maturity of the vulnerability.

Scans	Total	Fixes	Critical	High	Medium
3	368	60	6	35	101

Filter

Filter vulnerabilities

Name	Effective severity	CVSS	Fix available	Package	Package type	
CVE-2017-16997	Critical	9.3	Yes	glibc	OS	VIEW FIX
CVE-2015-20107	Critical	8	-	python3.5	OS	VIEW
CVE-2022-2068	Critical	10	-	openssl	OS	VIEW
CVE-2019-3462	Critical	9.3	Yes	apt	OS	VIEW FIX
CVE-2022-1292	Critical	10	Yes	openssl	OS	VIEW FIX
CVE-2019-19814	Critical	9.3	-	linux	OS	VIEW
CVE-2018-12931	High	7.2	-	linux	OS	VIEW
CVE-2019-0145	High	7.2	-	linux	OS	VIEW
CVE-2018-1000001	High	7.2	-	glibc	OS	VIEW
CVE-2017-18269	High	7.5	Yes	glibc	OS	VIEW FIX
CVE-2018-15686	High	7.2	Yes	systemd	OS	VIEW FIX
CVE-2022-1271	High	8.8	Yes	gzip	OS	VIEW FIX

自動掃描功能預設為啟用。請參閱 [Artifact Registry 設定](#)，瞭解如何開啟/關閉自動掃描功能。

步驟 4 · 約 0 分鐘

4. On-Demand Scanning

可能會有各種情況會要求您先執行掃描，才能將映像檔推送至存放區。舉例來說，容器開發人員可能會掃描映像檔並修正問題，然後再將程式碼推送至原始碼控管。下方的範例中，您將在本機建構及分析映像檔，然後根據結果採取行動。

建構映像檔

在這個步驟中，您會使用本機 Docker 將映像檔建構至本機快取。

```
docker build -t us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image .
```

掃描圖片

建構映像檔後，請要求掃描映像檔。掃描結果會儲存在中繼資料伺服器。工作完成後，中繼資料伺服器會提供結果的位置。

```
gcloud artifacts docker images scan \
  us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image \
  --format="value(response.scan)" > scan_id.txt
```

查看輸出檔案

請花點時間查看上一個步驟的輸出內容，這些內容會儲存在 scan_id.txt 檔案中。請注意中繼資料伺服器中掃描結果的報表位置。

```
cat scan_id.txt
```

查看詳細掃描結果

如要查看實際的掃描結果，請在輸出檔案中記下的報表位置使用 `list-vulnerabilities` 指令。

```
gcloud artifacts docker images list-vulnerabilities $(cat scan_id.txt)
```

輸出內容包含大量有關映像檔中所有安全漏洞的資料。

標記重大問題

人類很少直接使用儲存在報表的資料，通常是自動化程序使用這些結果。使用下列指令讀取報表詳細資料，並記錄是否發現任何重大安全漏洞

```
export SEVERITY=CRITICAL

gcloud artifacts docker images list-vulnerabilities $(cat scan_id.txt) --
format="value(vulnerability.effectiveSeverity)" | if grep -Fxq ${SEVERITY}; then echo "Failed
vulnerability check for ${SEVERITY} level"; else echo "No ${SEVERITY} Vulnerabilities found";
fi
```

這項指令的輸出內容如下

```
Failed vulnerability check for CRITICAL level
```

步驟 5 · 約 0 分鐘

5. 建構管道掃描

在本節中，您將建立自動化的建構管道，以打造容器映像檔、執行掃描，然後評估結果。如果未發現重大安全漏洞，系統就會將映像檔推送至存放區。如果發現重大安全漏洞，建構作業就會失敗並結束。

提供 Cloud Build 服務帳戶的存取權

Cloud Build 必須具備 On-Demand Scanning API 的存取權。使用下列指令提供存取權。

```
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com" \
  --role="roles/iam.serviceAccountUser"

gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com" \
  --role="roles/ondemandscanning.admin"
```

建立 Cloud Build 管道

下列指令會在目錄中建立 cloudbuild.yaml 檔案，供自動化程序使用。本範例的步驟僅限於容器建構程序。不過，實際上，除了容器步驟，您還會加入應用程式專用操作說明和測試。

使用下列指令建立檔案。

```

cat > ./cloudbuild.yaml << EOF
steps:

# build
- id: "build"
  name: 'gcr.io/cloud-builders/docker'
  args: ['build', '-t', 'us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-
repo/sample-image', '.']
  waitFor: ['-']

#Run a vulnerability scan at _SECURITY level
- id: scan
  name: 'gcr.io/cloud-builders/gcloud'
  entrypoint: 'bash'
  args:
  - '-c'
  - |
    (gcloud artifacts docker images scan \
    us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image \
    --location us \
    --format="value(response.scan)") > /workspace/scan_id.txt

#Analyze the result of the scan
- id: severity check
  name: 'gcr.io/cloud-builders/gcloud'
  entrypoint: 'bash'
  args:
  - '-c'
  - |
    gcloud artifacts docker images list-vulnerabilities \$(cat /workspace/scan_id.txt) \
    --format="value(vulnerability.effectiveSeverity)" | if grep -Fxq CRITICAL; \
    then echo "Failed vulnerability check for CRITICAL level" && exit 1; else echo "No
CRITICAL vulnerability found, congrats !" && exit 0; fi

#Retag
- id: "retag"
  name: 'gcr.io/cloud-builders/docker'
  args: ['tag', 'us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-
image', 'us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image:good']

#pushing to artifact registry
- id: "push"
  name: 'gcr.io/cloud-builders/docker'
  args: ['push', 'us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-
image:good']

images:
  - us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image
EOF

```

執行 CI 管道

提交建構作業進行處理，確認在發現重大嚴重性安全漏洞時，是否會建構失敗。

```
gcloud builds submit
```

查看建構失敗情形

由於映像檔含有重大安全漏洞，您剛提交的建構作業將會失敗。

在「[Cloud Build 記錄](#)」頁面中查看建構失敗情形

修正安全漏洞

更新 Dockerfile，使用不含重大安全漏洞的基本映像檔。

使用下列指令覆寫 Dockerfile，改用 Debian 10 映像檔：

```
cat > ./Dockerfile << EOF
from python:3.8-slim

# App
WORKDIR /app
COPY . ./

RUN pip3 install Flask==2.1.0
RUN pip3 install gunicorn==20.1.0

CMD exec gunicorn --bind :$PORT --workers 1 --threads 8 main:app

EOF
```

使用良好圖片執行 CI 程序

提交建構作業進行處理，確認在未發現重大嚴重性安全漏洞時，是否會建構成功。

```
gcloud builds submit
```

查看建構作業是否成功

您剛提交的建構作業會成功，因為更新後的映像檔不含重大安全漏洞。

在「[Cloud Build 記錄](#)」頁面中查看建構作業是否成功

查看掃描結果

在 Artifact Registry 查看正常映像檔

1. 在 Cloud 控制台開啟 [Artifact Registry](#)
2. 點選「artifact-scanning-repo」查看內容
3. 點選圖片詳細資料

- 4. 點選映像檔最新的摘要
 - 5. 點選映像檔的安全漏洞分頁標籤
-

步驟 6 · 約 0 分鐘

6. 簽署映像檔

建立驗證者附註

驗證者附註只是一小份資料，可做為所套用簽章類型的標籤。舉例來說，某則附註可能指示安全漏洞掃描，另一則可能會用於 QA 簽核。執行簽署流程時會參考附註。

建立記事

```
cat > ./vulnz_note.json << EOM
{
  "attestation": {
    "hint": {
      "human_readable_name": "Container Vulnerabilities attestation authority"
    }
  }
}
EOM
```

儲存附註

```
NOTE_ID=vulnz_note

curl -vvv -X POST \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $(gcloud auth print-access-token)" \
  --data-binary @./vulnz_note.json \
  "https://containeranalysis.googleapis.com/v1/projects/${PROJECT_ID}/notes/?
noteId=${NOTE_ID}"
```

驗證附註

```
curl -vvv \
  -H "Authorization: Bearer $(gcloud auth print-access-token)" \
  "https://containeranalysis.googleapis.com/v1/projects/${PROJECT_ID}/notes/${NOTE_ID}"
```

建立驗證者

驗證者會執行實際的映像檔簽署程序，並將附註的例項附加至映像檔，以供日後驗證。建立驗證者，以供日後使用。

建立驗證者

```
ATTESTOR_ID=vulnz-attestor

gcloud container binauthz attestors create $ATTESTOR_ID \
  --attestation-authority-note=$NOTE_ID \
  --attestation-authority-note-project=${PROJECT_ID}
```

檢查驗證者

```
gcloud container binauthz attestors list
```

請注意，最後一行會顯示 `NUM_PUBLIC_KEYS: 0`，您將在後續步驟中提供金鑰

此外，執行會產生映像檔的建構作業時，Cloud Build 會在專案中自動建立 `built-by-cloud-build` 驗證者。因此上述指令會傳回兩個驗證者：`vulnz-attestor` 和 `built-by-cloud-build`。映像檔建構完成後，Cloud Build 會自動簽署映像檔並建立驗證。

新增 IAM 角色

[二進位授權](#)服務帳戶需要查看驗證附註的權限。透過下列 API 呼叫提供存取權

```
PROJECT_NUMBER=$(gcloud projects describe "${PROJECT_ID}" --format="value(projectNumber)")

BINAUTHZ_SA_EMAIL="service-${PROJECT_NUMBER}@gcp-sa-binaryauthorization.iam.gserviceaccount.com"

cat > ./iam_request.json << EOM
{
  'resource': 'projects/${PROJECT_ID}/notes/${NOTE_ID}',
  'policy': {
    'bindings': [
      {
        'role': 'roles/containeranalysis.notes.occurrences.viewer',
        'members': [
          'serviceAccount:${BINAUTHZ_SA_EMAIL}'
        ]
      }
    ]
  }
}
EOM
```

使用檔案建立 IAM 政策

```
curl -X POST \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $(gcloud auth print-access-token)" \
  --data-binary @./iam_request.json \

"https://containeranalysis.googleapis.com/v1/projects/${PROJECT_ID}/notes/${NOTE_ID}:setIamPolicy"
```

新增 KMS 金鑰

驗證者需要密碼編譯金鑰才能附加附註，並提供可驗證的簽章。在本步驟中，您會在 KMS 中建立及儲存金鑰，供 Cloud Build 後續存取。

首先，新增一些環境變數來描述新金鑰

```
KEY_LOCATION=global
KEYRING=binauthz-keys
KEY_NAME=codelab-key
KEY_VERSION=1
```

建立金鑰環來保存一組金鑰

```
gcloud kms keyrings create "${KEYRING}" --location="${KEY_LOCATION}"
```

為驗證者建立新的非對稱簽署金鑰組

```
gcloud kms keys create "${KEY_NAME}" \
  --keyring="${KEYRING}" --location="${KEY_LOCATION}" \
  --purpose asymmetric-signing \
  --default-algorithm="ec-sign-p256-sha256"
```

Google Cloud 控制台的 [KMS 頁面](#) 應會顯示您的金鑰。

現在，透過 `gcloud binauthz` 指令將金鑰與驗證者建立關聯：

```
gcloud beta container binauthz attestors public-keys add \
  --attestor="${ATTESTOR_ID}" \
  --keyversion-project="${PROJECT_ID}" \
  --keyversion-location="${KEY_LOCATION}" \
  --keyversion-keyring="${KEYRING}" \
  --keyversion-key="${KEY_NAME}" \
  --keyversion="${KEY_VERSION}"
```

如果您再次列印授權單位清單，現在應該會看到已註冊的金鑰：

```
gcloud container binauthz attestors list
```

建立已簽署的驗證

到了這個階段，您已經設定了簽署映像檔的功能。使用先前建立的驗證者簽署您使用的容器映像檔

```
CONTAINER_PATH=us-central1-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image

DIGEST=$(gcloud container images describe ${CONTAINER_PATH}:latest \
  --format='get(image_summary.digest)')
```

現在，您可以使用 `gcloud` 建立驗證。這項指令會輸入您要用於簽署的金鑰詳細資料，以及要核准的特定容器映像檔

```
gcloud beta container binauthz attestations sign-and-create \
  --artifact-url="${CONTAINER_PATH}@${DIGEST}" \
  --attestor="${ATTESTOR_ID}" \
  --attestor-project="${PROJECT_ID}" \
  --keyversion-project="${PROJECT_ID}" \
  --keyversion-location="${KEY_LOCATION}" \
  --keyversion-keyring="${KEYRING}" \
  --keyversion-key="${KEY_NAME}" \
  --keyversion="${KEY_VERSION}"
```

以容器分析的用語來說，這會建立新例項，並附加至驗證者的附註。為確保一切運作正常，您可以列出驗證

```
gcloud container binauthz attestations list \
  --attestor=$ATTESTOR_ID --attestor-project=${PROJECT_ID}
```

步驟 7 · 約 0 分鐘

7. 使用 Cloud Build 簽署

您已啟用映像檔簽署功能，並手動使用驗證者簽署範例映像檔。在實務中，您會在自動化程序 (例如 CI/CD 管道) 中套用驗證。

在本節中，您將設定 [Cloud Build](#) 自動驗證映像檔。

角色

將二進位授權驗證者檢視者角色新增至 Cloud Build 服務帳戶：

```
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com \
  --role roles/binaryauthorization.attestorsViewer
```

將 Cloud KMS CryptoKey Signer/Verifier 角色新增至 Cloud Build 服務帳戶 (KMS 型簽署)：

```
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com \
  --role roles/cloudkms.signerVerifier
```

將容器分析註記連結者角色新增至 Cloud Build 服務帳戶：

```
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com \
  --role roles/containeranalysis.notes.attacher
```

準備自訂建構 Cloud Build 步驟

您將在 Cloud Build 中使用自訂建構步驟，簡化驗證程序。Google 提供的這個自訂建構步驟包含輔助函式，可簡化程序。在使用前，必須先將自訂建構步驟的程式碼建構成容器，並推送至 Cloud Build。請執行下列指令來進行這項操作：

```
git clone https://github.com/GoogleCloudPlatform/cloud-builders-community.git
cd cloud-builders-community/binauthz-attestation
gcloud builds submit . --config cloudbuild.yaml
cd ../../
rm -rf cloud-builders-community
```

在 cloudbuild.yaml 中新增簽署步驟

在這個步驟中，您將在先前建構的 Cloud Build 管道中新增驗證步驟。

1. 查看要新增的步驟。

僅供檢閱，請勿複製

```
#Sign the image only if the previous severity check passes
- id: 'create-attestation'
  name: 'gcr.io/${PROJECT_ID}/binauthz-attestation:latest'
  args:
    - '--artifact-url'
    - 'us-central1-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image'
    - '--attestor'
    - 'projects/${PROJECT_ID}/attestors/${ATTESTOR_ID}'
    - '--keyversion'
    -
    'projects/${PROJECT_ID}/locations/${KEY_LOCATION}/keyRings/${KEYRING}/cryptoKeys/${KEY_NAME}/cryptoKeyVersions/${KEY_VERSION}'
```

2. 使用更新的完整管道覆寫 cloudbuild.yaml 檔案。

```

cat > ./cloudbuild.yaml << EOF
steps:

# build
- id: "build"
  name: 'gcr.io/cloud-builders/docker'
  args: ['build', '-t', 'us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-
repo/sample-image', '.']
  waitFor: ['-']

#Run a vulnerability scan at _SECURITY level
- id: scan
  name: 'gcr.io/cloud-builders/gcloud'
  entrypoint: 'bash'
  args:
  - '-c'
  - |
    (gcloud artifacts docker images scan \
    us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image \
    --location us \
    --format="value(response.scan)") > /workspace/scan_id.txt

#Analyze the result of the scan
- id: severity check
  name: 'gcr.io/cloud-builders/gcloud'
  entrypoint: 'bash'
  args:
  - '-c'
  - |
    gcloud artifacts docker images list-vulnerabilities \$(cat /workspace/scan_id.txt) \
    --format="value(vulnerability.effectiveSeverity)" | if grep -Fxq CRITICAL; \
    then echo "Failed vulnerability check for CRITICAL level" && exit 1; else echo "No
CRITICAL vulnerability found, congrats !" && exit 0; fi

#Retag
- id: "retag"
  name: 'gcr.io/cloud-builders/docker'
  args: ['tag', 'us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-
image', 'us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image:good']

#pushing to artifact registry
- id: "push"
  name: 'gcr.io/cloud-builders/docker'
  args: ['push', 'us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-
image:good']

#Sign the image only if the previous severity check passes
- id: 'create-attestation'
  name: 'gcr.io/${PROJECT_ID}/binauthz-attestation:latest'
  args:

```

```
- '--artifact-url'
- 'us-central1-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image:good'
- '--attestor'
- 'projects/${PROJECT_ID}/attestors/${ATTESTOR_ID}'
- '--keyversion'
-
'projects/${PROJECT_ID}/locations/${KEY_LOCATION}/keyRings/${KEYRING}/cryptoKeys/${KEY_NAME}/crypto
KeyVersions/${KEY_VERSION}'

images:
- us-central1-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image:good
EOF
```

執行建構作業

```
gcloud builds submit
```

在 Cloud Build 記錄中查看建構作業

開啟 Cloud 控制台的 [Cloud Build 記錄頁面](#)，查看最新建構作業和建構步驟是否成功執行。

步驟 8 · 約 0 分鐘

8. 許可控制政策

二進位授權是 GKE 和 Cloud Run 的一項功能，可在允許執行容器映像檔前驗證規則。無論是來自可信任的 CI/CD 管道，還是使用者手動嘗試部署映像檔，只要有執行映像檔的要求，系統就會執行驗證。與單獨檢查 CI/CD 管道相比，這項功能保護執行階段環境的效果更好。

若要瞭解這項功能，需修改預設 GKE 政策，強制執行嚴格的授權規則。

建立 GKE 叢集

建立 GKE 叢集：

```
gcloud beta container clusters create binauthz \
  --zone us-central1-a \
  --binauthz-evaluation-mode=PROJECT_SINGLETON_POLICY_ENFORCE
```

允許 Cloud Build 部署至這個叢集：

```
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com" \
  --role="roles/container.developer"
```

允許所有政策

請先驗證預設政策狀態，以及您部署任何映像檔的能力

1. 查看現有政策

```
gcloud container binauthz policy export
```

2. 請注意，強制執行政策已設為 `ALWAYS_ALLOW`

```
evaluationMode: ALWAYS_ALLOW
```

3. 部署範例，確認您可以部署任何內容

```
kubectl run hello-server --image gcr.io/google-samples/hello-app:1.0 --port 8080
```

4. 確認部署作業是否正常運作

```
kubectl get pods
```

您會看到下列輸出內容

NAME	READY	STATUS	RESTARTS	AGE
hello-server	1/1	Running	0	13s

5. 刪除部署作業

```
kubectl delete pod hello-server
```

拒絕所有政策

現在更新政策，禁止顯示所有映像檔。

1. 將現行政策匯出為可編輯的檔案

```
gcloud container binauthz policy export > policy.yaml
```

2. 變更政策

在文字編輯器中，將 `evaluationMode` 從 `ALWAYS_ALLOW` 變更為 **`ALWAYS_DENY`**。

```
edit policy.yaml
```

政策 YAML 檔案應如下所示：

```
globalPolicyEvaluationMode: ENABLE
defaultAdmissionRule:
  evaluationMode: ALWAYS_DENY
  enforcementMode: ENFORCED_BLOCK_AND_AUDIT_LOG
name: projects/PROJECT_ID/policy
```

3. 開啟終端機並套用新政策，然後稍候片刻，等待變更生效

```
gcloud container binauthz policy import policy.yaml
```

4. 嘗試部署範例工作負載

```
kubectl run hello-server --image gcr.io/google-samples/hello-app:1.0 --port 8080
```

5. 部署作業失敗，並顯示下列訊息

```
Error from server (VIOLATES_POLICY): admission webhook "imagepolicywebhook.image-policy.k8s.io" denied the request: Image gcr.io/google-samples/hello-app:1.0 denied by Binary Authorization default admission rule. Denied by always_deny admission rule
```

將政策還原為允許所有項目

繼續進行下一節的操作之前，請務必還原政策變更

1. 變更政策

在文字編輯器中，將 `evaluationMode` 從 `ALWAYS_DENY` 變更為 **`ALWAYS_ALLOW`**。

```
edit policy.yaml
```


政策 YAML 檔案應如下所示：

```
globalPolicyEvaluationMode: ENABLE
defaultAdmissionRule:
  evaluationMode: ALWAYS_ALLOW
  enforcementMode: ENFORCED_BLOCK_AND_AUDIT_LOG
name: projects/PROJECT_ID/policy
```

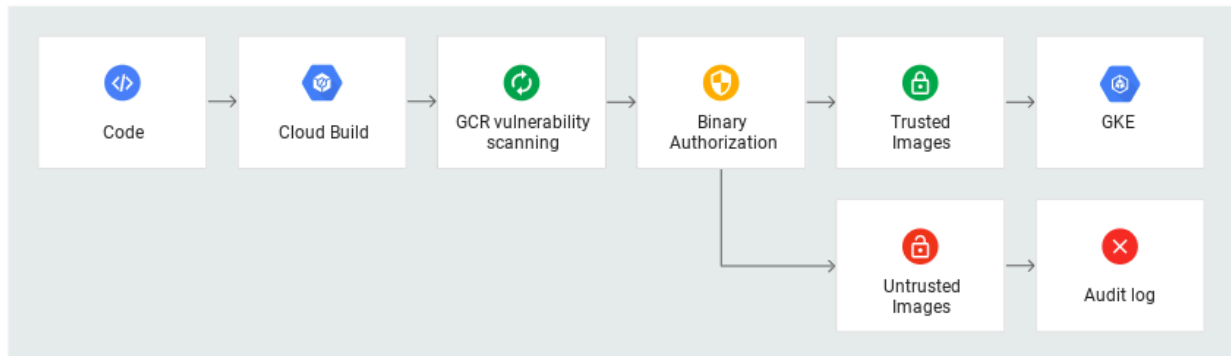
2. 套用還原的政策

```
gcloud container binauthz policy import policy.yaml
```

步驟 9 · 約 0 分鐘

9. 封鎖 GKE 中的安全漏洞

在本節中，您將整合目前所學內容，透過 Cloud Build 實作 CI/CD 管道，掃描映像檔，然後檢查安全漏洞，再簽署映像檔並嘗試部署。GKE 會使用二進位授權驗證映像檔是否具有安全漏洞掃描的簽章，再允許映像檔執行。



更新 GKE 政策以要求驗證

在 GKE BinAuth 政策中新增 clusterAdmissionRules，要求映像檔必須由驗證者簽署

使用下列指令，以更新版設定覆寫政策。

```
COMPUTE_ZONE=us-central1-a

cat > binauth_policy.yaml << EOM
defaultAdmissionRule:
  enforcementMode: ENFORCED_BLOCK_AND_AUDIT_LOG
  evaluationMode: ALWAYS_DENY
globalPolicyEvaluationMode: ENABLE
clusterAdmissionRules:
  ${COMPUTE_ZONE}.binauthz:
    evaluationMode: REQUIRE_ATTESTATION
    enforcementMode: ENFORCED_BLOCK_AND_AUDIT_LOG
    requireAttestationsBy:
      - projects/${PROJECT_ID}/attestors/vulnz-attestor
EOM
```

套用政策

```
gcloud beta container binauthz policy import binauth_policy.yaml
```

嘗試部署未簽署的映像檔

使用下列指令，為先前建構的應用程式建立部署描述元。這裡使用的映像檔是您先前建構的映像檔，其中含有重大安全漏洞，且不含簽署的認證。

GKE 准入控制器必須知道要部署的確切映像檔，才能持續驗證簽章。如要達成這個目標，您需要使用圖片摘要，而不是簡單的標記。

取得不良映像檔的摘要

```
CONTAINER_PATH=us-central1-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image
```

```
DIGEST=$(gcloud container images describe ${CONTAINER_PATH}:bad \
  --format='get(image_summary.digest)')
```

在 Kubernetes 設定中使用摘要

```
cat > deploy.yaml << EOM
apiVersion: v1
kind: Service
metadata:
  name: deb-httpd
spec:
  selector:
    app: deb-httpd
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deb-httpd
spec:
  replicas: 1
  selector:
    matchLabels:
      app: deb-httpd
  template:
    metadata:
      labels:
        app: deb-httpd
    spec:
      containers:
        - name: deb-httpd
          image: ${CONTAINER_PATH}@${DIGEST}
          ports:
            - containerPort: 8080
          env:
            - name: PORT
              value: "8080"

EOM
```

嘗試將應用程式部署至 GKE

```
kubectl apply -f deploy.yaml
```

檢視[控制台中的工作負載](#)，並記下指出部署作業遭拒絕的錯誤：

```
No attestations found that were valid and signed by a key trusted by the attestor
```

部署已簽署的映像檔

取得不良映像檔的摘要

```
CONTAINER_PATH=us-central1-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image

DIGEST=$(gcloud container images describe ${CONTAINER_PATH}:good \
  --format='get(image_summary.digest)')
```

在 Kubernetes 設定中使用摘要

```
cat > deploy.yaml << EOM
apiVersion: v1
kind: Service
metadata:
  name: deb-httpd
spec:
  selector:
    app: deb-httpd
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deb-httpd
spec:
  replicas: 1
  selector:
    matchLabels:
      app: deb-httpd
  template:
    metadata:
      labels:
        app: deb-httpd
    spec:
      containers:
        - name: deb-httpd
          image: ${CONTAINER_PATH}@${DIGEST}
          ports:
            - containerPort: 8080
          env:
            - name: PORT
              value: "8080"

EOM
```

將應用程式部署至 GKE

```
kubectl apply -f deploy.yaml
```

檢視[控制台中的工作負載](#)，並記下映像檔已成功部署。

步驟 10 · 約 0 分鐘

10. 恭喜！

恭喜，您已完成本程式碼研究室！

涵蓋內容：

- 如何啟用自動掃描功能
- 如何執行 On-Demand Scanning
- 如何在建構管道中整合掃描功能
- 如何簽署核准的圖片
- 如何使用 GKE 准入控制器封鎖映像檔
- 如何設定 GKE，只允許已簽署的核准映像檔

後續步驟：

- [保護部署至 Cloud Run 和 Google Kubernetes Engine 的映像檔 | Cloud Build 說明文件](#)
- [快速入門導覽課程：為 GKE | Google Cloud 設定二進位授權政策](#)

清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本教學課程所用資源的費用，請刪除含有相關資源的專案，或者保留專案但刪除個別資源。

刪除專案

如要避免付費，最簡單的方法就是刪除您為了本教學課程所建立的專案。
